

Kubernetes-02

Tasks:

1. **Create a Simple Pod Using YAML Task:** Write a YAML file to create a Pod named firstpod with an nginx container. Verify the Pod creation using `kubectl get pods` and check the logs of the container using `kubectl logs firstpod`.
2. **Set Environment Variables in a Pod Task:** Modify the YAML file to include environment variables `myname: sabair` and `City: Hyderabad`. Deploy the Pod and use `kubectl exec <pod_name> -- env` to check if the environment variables are set properly.
3. **Deploy a Pod with Commands (Args) in YAML Task:** Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use `kubectl describe pod` to verify the args are correctly passed to the container.
4. **Create a Pod with Two Containers Task:** Create a YAML file to define a Pod with two nginx containers inside. Use `kubectl exec` to access both containers and verify that both containers can communicate through the same network (e.g., using `telnet` between them).
5. **Set Up an Init Container in a Pod Task:** Modify the YAML to include an init container that sleeps for 30 seconds before the main containers start. Verify the init container's execution using `kubectl describe pod` and check the logs to confirm its completion.
6. **Run a Dry Run Command to Generate YAML Task:** Use the `kubectl run nginx --image=nginx --dry-run=client -o yaml` command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.
7. **Use kubectl apply vs kubectl create Task:** Create a YAML file to define a Pod. First, deploy it using `kubectl create -f <file_name>.yaml` and then modify the YAML (e.g., change the image version). Use `kubectl apply` to redeploy and verify the difference between both commands.
8. **Edit an Existing Pod Configuration Task:** Use `kubectl edit pod <pod_name>` to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using `kubectl exec`.
9. **Expose a Pod Using a Service Task:** Create a YAML file to expose your firstpod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using `kubectl get svc`.
10. **Pod with Resource Limits and Requests Task:** Add resource requests and limits to the containers in your YAML file. Specify CPU and memory requests/limits for both containers and deploy the Pod. Use `kubectl describe pod` to verify if the resource configurations are correctly applied.

TASK 1 — Create Simple Pod (firstpod)

1. Create a Simple Pod Using YAML Task: Write a YAML file to create a Pod named firstpod with an nginx container. Verify the Pod creation using `kubectl get pods` and check the logs of the container using `kubectl logs firstpod`.

In your master

Step 1 — Create YAML file

In your terminal:

```
vi firstpod.yml
```

A terminal window with a black background and green text. The prompt is [ubuntu@master:~\$]. The command cat firstpod.yml is entered, followed by the contents of the file: apiVersion: v1, kind: Pod, metadata: name: firstpod, spec: containers: - name: nginx-container, image: nginx, ports: - containerPort: 80. The prompt returns to [ubuntu@master:~\$].

```
[ubuntu@master:~$ cat firstpod.yml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: 80
ubuntu@master:~$
```

Step 2 — Deploy the Pod

```
kubectl apply -f firstpod.yml
```

Step 3 — Verify Pod

```
kubectl get pods
```

```
kubectl get pods -o wide
```

You should see status:

```
firstpod    Running
```

```
ubuntu@master:~$ kubectl apply -f firstpod.yml
pod/firstpod created
ubuntu@master:~$ kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
firstpod    1/1     Running   0           12s
ubuntu@master:~$ kubectl get pods -o wide
NAME        READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
firstpod    1/1     Running   0           39s   192.168.189.66 worker2     <none>           <none>
```

Step 4 — Check Logs

```
kubectl logs firstpod
```

```
ubuntu@master:~$ kubectl logs firstpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/17 09:26:47 [notice] 1#1: using the "epoll" event method
2026/02/17 09:26:47 [notice] 1#1: nginx/1.29.5
2026/02/17 09:26:47 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/17 09:26:47 [notice] 1#1: OS: Linux 6.14.0-1018-aws
2026/02/17 09:26:47 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/17 09:26:47 [notice] 1#1: start worker processes
2026/02/17 09:26:47 [notice] 1#1: start worker process 30
2026/02/17 09:26:47 [notice] 1#1: start worker process 31
ubuntu@master:~$ cat firstpod.yml
```

From your **master node**, try accessing the pod directly:

```
curl 192.168.189.66
```

```
ubuntu@master:~$ curl 192.168.189.66
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@master:~$
```

Why browser won't open?

Because this IP is **cluster private network**

Only nodes inside Kubernetes can access it — not your laptop.

Later we create **Service** → then public access works.

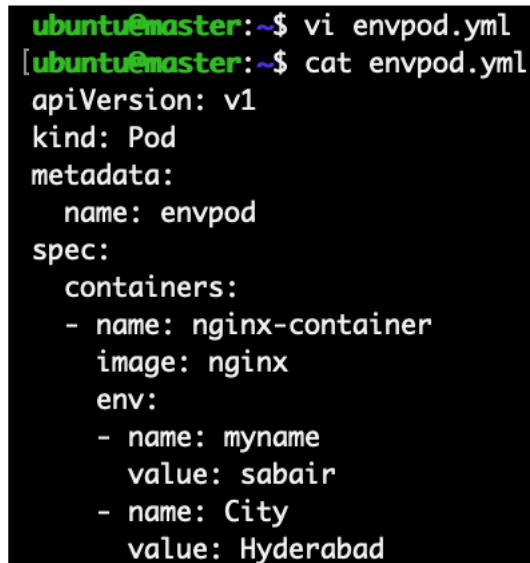
TASK 2 — Pod with Environment Variables

2. Set Environment Variables in a Pod Task: Modify the YAML file to include environment variables myname: sabair and City: Hyderabad. Deploy the Pod and use `kubectl exec <pod_name> -- env` to check if the environment variables are set properly.

We will create a new pod and check if variables exist inside container.

Step 1 — Create YAML

```
vi envpod.yml
```

A terminal window with a black background and green text. It shows the command 'vi envpod.yml' being executed, followed by 'cat envpod.yml' which displays the contents of the file. The file is a Kubernetes Pod manifest with a single container named 'nginx-container' using the 'nginx' image. Two environment variables are defined: 'myname' with value 'sabair' and 'City' with value 'Hyderabad'.

```
ubuntu@master:~$ vi envpod.yml
[ubuntu@master:~$ cat envpod.yml
apiVersion: v1
kind: Pod
metadata:
  name: envpod
spec:
  containers:
  - name: nginx-container
    image: nginx
    env:
    - name: myname
      value: sabair
    - name: City
      value: Hyderabad
```

Step 2 — Deploy Pod

```
kubectl apply -f envpod.yml
```

Step 3 — Verify Pod Running

```
kubectl get pods
```

```
kubectl get pods -o wide
```

```
ubuntu@master:~$ kubectl apply -f envpod.yml
pod/envpod created
ubuntu@master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
envpod	1/1	Running	0	9s
firstpod	1/1	Running	0	15m

```
HOME=/root
ubuntu@master:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
envpod	1/1	Running	0	91s	192.168.235.131	worker1	<none>	<none>
firstpod	1/1	Running	0	17m	192.168.189.66	worker2	<none>	<none>

```
ubuntu@master:~$
```

Step 4 — Validate Environment Variables (Main Proof)

```
kubectl exec envpod -- env
```

```
ubuntu@master:~$ kubectl exec envpod -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=envpod
NGINX_VERSION=1.29.5
NJS_VERSION=0.9.5
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
myname=sabair
City=Hyderabad
NGINX_SERVICE_PORT_80_TCP_PROTO=tcp
NGINX_SERVICE_PORT_80_TCP_PORT=80
NGINX_SERVICE_PORT_80_TCP_ADDR=10.107.18.212
KUBERNETES_PORT_443_TCP_PROTO=tcp
NGINX_SERVICE_SERVICE_PORT=80
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PORT=443
NGINX_SERVICE_PORT_80_TCP=tcp://10.107.18.212:80
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
NGINX_SERVICE_SERVICE_HOST=10.107.18.212
NGINX_SERVICE_PORT=tcp://10.107.18.212:80
KUBERNETES_SERVICE_HOST=10.96.0.1
HOME=/root
```

TASK 3 — Pod with Args (Sleep 50 seconds)

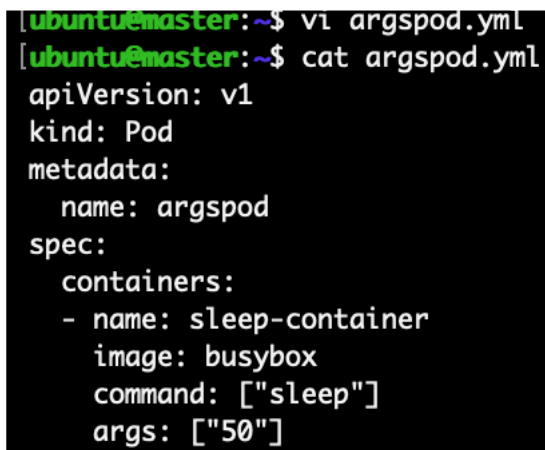
3. Deploy a Pod with Commands (Args) in YAML Task:
Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use kubectl describe pod to verify the args are correctly passed to the container.

Now we create a pod that does not run nginx — it only runs a command:

—> Container will start → sleep 50 sec → stop

Step 1 — Create YAML

Vi argspod.yml

A terminal window with a black background and green text. It shows two commands being executed: 'vi argspod.yml' and 'cat argspod.yml'. The output of the second command displays a YAML configuration for a Pod named 'argspod'.

```
[ubuntu@master:~]$ vi argspod.yml
[ubuntu@master:~]$ cat argspod.yml
apiVersion: v1
kind: Pod
metadata:
  name: argspod
spec:
  containers:
  - name: sleep-container
    image: busybox
    command: ["sleep"]
    args: ["50"]
```

Step 2 — Deploy

kubectl apply -f argspod.yml

Step 3 — Watch Pod Behaviour

Immediately run:

kubectl get pods -w

```
[ubuntu@master:~$ cat argspod.yml
apiVersion: v1
kind: Pod
metadata:
  name: argspod
spec:
  containers:
  - name: sleep-container
    image: busybox
    command: ["sleep"]
    args: ["50"]
```

```
ubuntu@master:~$ kubectl apply -f argspod.yml
pod/argspod created
```

```
ubuntu@master:~$ kubectl get pods -w
NAME          READY   STATUS    RESTARTS   AGE
argspod        1/1     Running   0           7s
envpod         1/1     Running   0          6m29s
firstpod       1/1     Running   0          22m
```

You should observe:

```
argspod      Running
argspod      Completed    (after 50 sec)
Press CTRL + C to stop watching.
```

```
ubuntu@master:~$ kubectl apply -f argspod.yml
pod/argspod created
ubuntu@master:~$ kubectl get pods -w
NAME          READY   STATUS    RESTARTS   AGE
argspod        1/1     Running   0           7s
envpod         1/1     Running   0          6m29s
firstpod       1/1     Running   0          22m
argspod        0/1     Completed 0           52s
argspod        1/1     Running   1 (2s ago)  53s
```

Step 4 — Validate (IMPORTANT)

```
kubectl describe pod argspod
```

```
argspod 1/1 Running 2 (15s ago) 1155
^Cubuntu@master:~$ kubectl describe pod argspod
Name:         argspod
Namespace:    default
Priority:      0
Service Account: default
Node:         worker1/172.31.9.88
Start Time:   Tue, 17 Feb 2026 09:48:57 +0000
Labels:       <none>
Annotations:  cni.projectcalico.org/containerID: 6bdcfbd606c4418538707670c28f3d740cc9a22c75547597be935ea
              cni.projectcalico.org/podIP: 192.168.235.132/32
              cni.projectcalico.org/podIPs: 192.168.235.132/32
Status:       Running
IP:           192.168.235.132
IPs:          IP: 192.168.235.132
Containers:
  sleep-container:
    Container ID:  containerd://e062a7672c072ffe7d675399ab49406c0bacb253d0de26bfc63b2c2aaabee8f5
    Image:          busybox
    Image ID:       docker.io/library/busybox@sha256:b3255e7dfbcd10cb367af0d409747d511aeb66dfac98cf30e97e87e4
    Port:           <none>
    Host Port:      <none>
    Command:
      sleep
    Args:
      50
    State:          Running
      Started:      Tue, 17 Feb 2026 09:50:55 +0000
    Last State:     Terminated
      Reason:       Completed
      Exit Code:    0
      Started:      Tue, 17 Feb 2026 09:49:49 +0000
      Finished:     Tue, 17 Feb 2026 09:50:39 +0000
    Ready:          True
    Restart Count:  2
    Environment:    <none>
```

TASK-4 — Pod with Two Containers

4. Create a Pod with Two Containers Task: Create a YAML file to define a Pod with two nginx containers inside. Use kubectl exec to access both containers and verify that both containers can communicate through the same network (e.g., using telnet between them).

Here we prove an important concept:

Containers inside same Pod share SAME IP + SAME network (localhost works between them)

We must run second nginx on different port.

Step 1 — Create YAML

Vi twocontainer.yml

```
[ubuntu@master:~]$ vi twocontainer.yml
[ubuntu@master:~]$ cat twocontainer.yml
apiVersion: v1
kind: Pod
metadata:
  name: twocontainer
spec:
  containers:
  - name: nginx-a
    image: nginx
    ports:
    - containerPort: 80

  - name: nginx-b
    image: nginx
    command: ["/bin/sh", "-c"]
    args:
    - sed -i 's/80/81/g' /etc/nginx/conf.d/default.conf && nginx -g 'daemon off;'
    ports:
    - containerPort: 81
```

Step 2 — Deploy Pod

```
kubectl apply -f twocontainer.yml
```

Check:

```
kubectl get pods
```

You must see:

```
twocontainer    2/2    Running
```

→ 2/2 means both containers started

```
ubuntu@master:~$ kubectl apply -f twocontainer.yml
pod/twocontainer created
ubuntu@master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
argspod	0/1	CrashLoopBackOff	5 (2m23s ago)	10m
envpod	1/1	Running	0	16m
firstpod	1/1	Running	0	32m
twocontainer	2/2	Running	1 (3s ago)	6s

```
ubuntu@master:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
argspod	0/1	CrashLoopBackOff	6 (86s ago)	12m	192.168.235.132	worker1	<none>	<none>
envpod	1/1	Running	0	19m	192.168.235.131	worker1	<none>	<none>
firstpod	1/1	Running	0	35m	192.168.189.66	worker2	<none>	<none>
twocontainer	2/2	Running	0	5s	192.168.189.68	worker2	<none>	<none>

```
ubuntu@master:~$
```

Now test communication

Get IP:

```
kubectl get pod twocontainer -o wide
```



```

twocontainer 2/2 Running 0 2m21s 192.168.189.68 worker2 <none> <none>
ubuntu@master:~$ kubectl get pod twocontainer -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
twocontainer 2/2 Running 0 3m1s 192.168.189.68 worker2 <none> <none>
ubuntu@master:~$

```

Verify Communication using telnet

Step 3 — Enter first container

```
kubectl exec -it twocontainer -c nginx-a -- bash
```

Step 2 — Install telnet

```

ubuntu@master:~$ kubectl exec -it twocontainer -c nginx-a -- bash
root@twocontainer:/# apt update
apt install telnet -y
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9670 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [108 kB]
Fetched 10.0 MB in 6s (1730 kB/s)
All packages are up to date.
Installing:
telnet

Installing dependencies:
inetutils-telnet netbase

Summary:
Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 0
Download size: 186 kB
Space needed: 377 kB / 15.4 GB available

```

Inside container:

```

apt update
apt install telnet -y

```

Step 3 — Connect to second container

(remember nginx-b runs on port 81)

```
telnet 192.168.189.68 81
```

```
Setting up telnet (0.17.12.0-3faeb15a1) ...  
root@twocontainer:/# telnet 192.168.189.68 81  
Trying 192.168.189.68...  
Connected to 192.168.189.68.  
Escape character is '^['.  
Connection closed by foreign host.  
root@twocontainer:/#
```

TASK-5 — Init Container (sleep before main container)

5. Set Up an Init Container in a Pod Task: Modify the YAML to include an init container that sleeps for 30 seconds before the main containers start. Verify the init container's execution using `kubectl describe pod` and check the logs to confirm its completion.

Goal:

Main container should start ONLY after init container finishes.

Step 1 — Create YAML

```
vi initpod.yml
```

```
exit
ubuntu@master:~$ vi initpod.yml
[ubuntu@master:~$ cat initpod.yml
apiVersion: v1
kind: Pod
metadata:
  name: initpod
spec:
  initContainers:
  - name: init-sleep
    image: busybox
    command: ["sh", "-c", "echo Initializing... && sleep 30"]

  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80

ubuntu@master:~$
```

Step 2 — Deploy

```
kubectl apply -f initpod.yml
```

```
ubuntu@master:~$ kubectl apply -f initpod.yml
pod/initpod created
ubuntu@master:~$ kubectl get pods -w
```

Step 3 — Watch Behaviour

```
kubectl get pods -w
```

You will observe:

```
initpod    Init:0/1
(wait 30 sec)
initpod    Running
Press CTRL + C
```

```
ubuntu@master:~$ kubectl apply -f initpod.yml
pod/initpod created
ubuntu@master:~$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
argspod	1/1	Running	8 (5m12s ago)	22m
envpod	1/1	Running	0	29m
firstpod	1/1	Running	0	44m
initpod	0/1	Init:0/1	0	6s
twocontainer	2/2	Running	0	9m53s

(wait 30 sec)

```
initpod    Running
```

Press **CTRL + C**

```
firstpod    1/1    Running    0    44m
initpod     0/1    Init:0/1   0    6s
twocontainer 2/2    Running    0    9m53s
initpod     0/1    PodInitializing 0    32s
initpod     1/1    Running    0    33s
```

Step 4 — Validate Init Container

```
kubectl describe pod initpod
```

```

^Cubuntu@master:~$ kubectl describe pod initpod
Name:          initpod
Namespace:     default
Priority:       0
Service Account: default
Node:          worker1/172.31.9.88
Start Time:    Tue, 17 Feb 2026 10:11:35 +0000
Labels:        <none>
Annotations:   cni.projectcalico.org/containerID: 5c35d33c98afb84350f580
               cni.projectcalico.org/podIP: 192.168.235.133/32
               cni.projectcalico.org/podIPs: 192.168.235.133/32
Status:        Running
IP:            192.168.235.133
IPs:
  IP: 192.168.235.133
Init Containers:
  init-sleep:
    Container ID:  containerd://81a0056cd18061d8f793db55304c5ec433744bd3a41
    Image:          busybox
    Image ID:       docker.io/library/busybox@sha256:b3255e7dfbcd10cb367af0d
    Port:          <none>
    Host Port:     <none>
    Command:
      sh
      -c
      echo Initializing... && sleep 30
    State:          Terminated
      Reason:       Completed
      Exit Code:    0
      Started:      Tue, 17 Feb 2026 10:11:36 +0000
      Finished:     Tue, 17 Feb 2026 10:12:06 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:

```

Also check logs:

```
kubectl logs initpod -c init-sleep
```

```

Normal      Successful   2/3      kubelet      Started container in
ubuntu@master:~$ kubectl logs initpod -c init-sleep
Initializing...
ubuntu@master:~$

```

TASK-6 — Generate YAML using Dry-Run

6. Run a Dry Run Command to Generate YAML Task: Use the `kubectl run nginx --image=nginx --dry-run=client -o yaml` command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.

Here you **don't** write **YAML manually** — Kubernetes creates it for you.

Step 1 — Generate YAML automatically

```
kubectl run nginx --image=nginx --dry-run=client -o yaml > dryrun.yaml
```

Now check file:

```
cat dryrun.yaml
```

You will see a full pod definition created by Kubernetes.

```
Initializing...
ubuntu@master:~$ kubectl run nginx --image=nginx --dry-run=client -o yaml > dryrun.yaml
[ubuntu@master:~$ ct dryrun.yaml
ct: command not found
[ubuntu@master:~$ cat dryrun.yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
ubuntu@master:~$
```

Step 2 — Modify YAML (examiner always asks this)

Open file:

```
Vi dryrun.yml
```

Add labels + environment variable

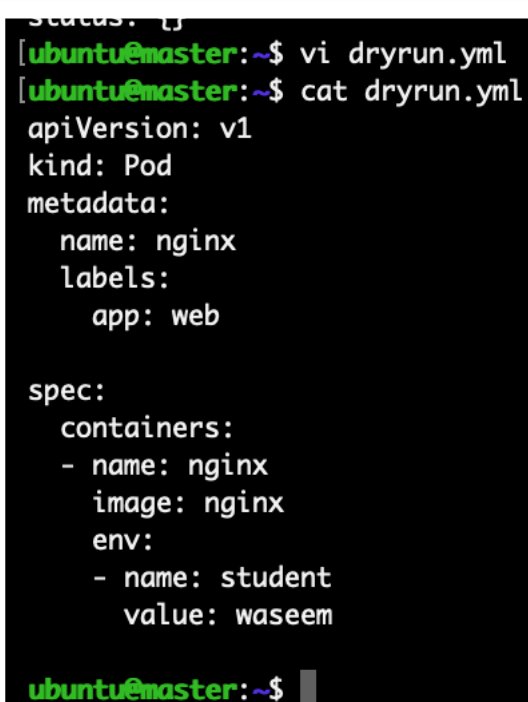
Modify metadata section:

```
metadata:
  name: nginx
  labels:
    app: web
```

Add env inside container:

```
env:
- name: student
  value: waseem
```

Save.

A terminal window with a black background and green text. The prompt is [ubuntu@master:~\$]. The user enters 'vi dryrun.yml' and then 'cat dryrun.yml'. The output shows a YAML configuration for a Pod. The 'metadata' section is updated with 'name: nginx' and 'labels: app: web'. The 'spec' section is updated with 'containers: - name: nginx, image: nginx, env: - name: student, value: waseem'. The prompt returns to [ubuntu@master:~\$].

```
status: {}
[ubuntu@master:~$ vi dryrun.yml
[ubuntu@master:~$ cat dryrun.yml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: web

spec:
  containers:
  - name: nginx
    image: nginx
    env:
    - name: student
      value: waseem

ubuntu@master:~$
```

Now Deploy It

Run:

```
kubectl apply -f dryrun.yml
```

```
ubuntu@master:~$ kubectl apply -f dryrun.yml
pod/nginx created
ubuntu@master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
argspod	0/1	CrashLoopBackOff	10 (29s ago)	35m
envpod	1/1	Running	0	42m
firstpod	1/1	Running	0	58m
initpod	1/1	Running	0	13m
nginx	1/1	Running	0	14s
twocontainer	2/2	Running	0	23m

```
ubuntu@master:~$
```

Validate Environment Variable (Main Proof)

```
kubectl exec nginx -- env
```

Scroll output and confirm:

```
twocontainer 2/2 Running 0 23m
ubuntu@master:~$ kubectl exec nginx -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=nginx
NGINX_VERSION=1.29.5
NJS_VERSION=0.9.5
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
student=waseem
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
NGINX_SERVICE_SERVICE_HOST=10.107.18.212
NGINX_SERVICE_PORT_80_TCP_ADDR=10.107.18.212
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
NGINX_SERVICE_PORT=tcp://10.107.18.212:80
NGINX_SERVICE_SERVICE_PORT=80
NGINX_SERVICE_PORT_80_TCP=tcp://10.107.18.212:80
NGINX_SERVICE_PORT_80_TCP_PROTO=tcp
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT_443_TCP_PORT=443
NGINX_SERVICE_PORT_80_TCP_PORT=80
HOME=/root
ubuntu@master:~$
```


student=waseem

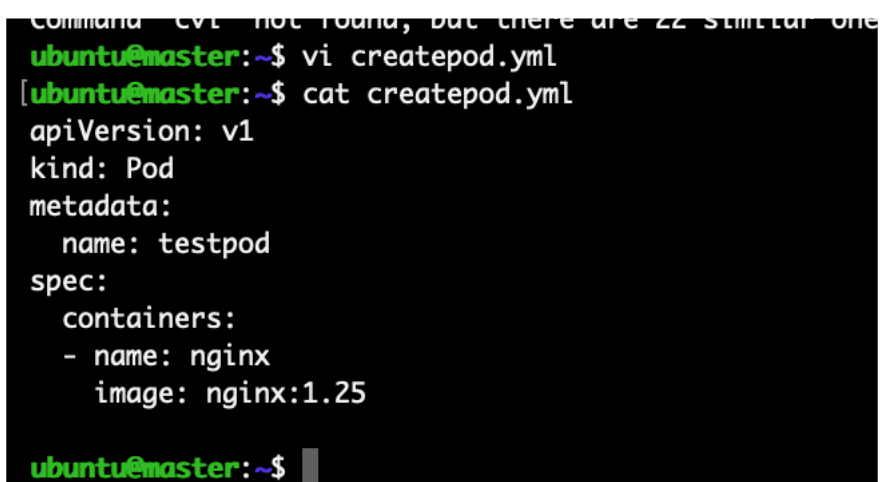
TASK-7 – kubectl create vs kubectl apply

7. Use kubectl apply vs kubectl create Task: Create a YAML file to define a Pod. First, deploy it using kubectl create -f <file_name>.yml and then modify the YAML (e.g., change the image version). Use kubectl apply to redeploy and verify the difference between both commands.

We will prove the difference practically.

Step 1 — Create a new pod YAML

```
vi createpod.yml
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The user enters 'vi createpod.yml'. The prompt changes to '[ubuntu@master:~\$' and the user enters 'cat createpod.yml'. The output is a YAML file defining a Pod: 'apiVersion: v1', 'kind: Pod', 'metadata: name: testpod', 'spec: containers: - name: nginx, image: nginx:1.25'. The prompt returns to 'ubuntu@master:~\$' with a cursor.

```
command 'vi' not found, but there are 22 similar ones
ubuntu@master:~$ vi createpod.yml
[ubuntu@master:~$ cat createpod.yml
apiVersion: v1
kind: Pod
metadata:
  name: testpod
spec:
  containers:
  - name: nginx
    image: nginx:1.25
ubuntu@master:~$
```

Step 2 — Deploy using CREATE

```
kubectl create -f createpod.yml
```

Check :

```
Kubectl get pods
```

```
ubuntu@master:~$ kubectl create -f createpod.yml
pod/testpod created
[ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
argspod       0/1     CrashLoopBackOff   11 (2m10s ago)    43m
envpod        1/1     Running         0           50m
firstpod      1/1     Running         0           65m
initpod       1/1     Running         0           21m
nginx         1/1     Running         0           8m
testpod       1/1     Running         0           16s
twocontainer  2/2     Running         0           30m
ubuntu@master:~$
```

Step 3 — Now MODIFY YAML

Edit file:

```
Vi createpod.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: testpod
spec:
  containers:
  - name: nginx
    image: nginx:alpine
```

Change image:

```
nginx:1.25 → nginx:alpine  
Save.
```

Step 4 — Apply changes using APPLY

```
kubectl apply -f createpod.yml
```

Step 5 — Verify update

```
kubectl describe pod testpod | grep Image  
You should now see:
```

```
Image: nginx:alpine
```

```
ubuntu@master:~$ vi createpod.yml  
ubuntu@master:~$ kubectl apply -f createpod.yml  
Warning: resource pods/testpod is missing the kubectl.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be  
nnotation will be patched automatically.  
pod/testpod configured  
ubuntu@master:~$ kubectl describe pod testpod | grep Image  
Image: nginx:alpine  
Image ID: docker.io/library/nginx@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8d7702ba  
Normal Pulled 3m21s kubelet Successfully pulled image "nginx:1.25" in 2.483s (2.484s including waiting). Image size: 71005258 bytes.  
Normal Pulled 8s kubelet Successfully pulled image "nginx:alpine" in 1.39s (1.39s including waiting). Image size: 25987617 bytes.  
ubuntu@master:~$
```

What you just proved

Command	Behaviour
create	only creates resource
apply	creates + updates existing resource

Do these steps and paste output of:

```
kubectl describe pod testpod | grep Image
```

Command	What it does
<code>kubectl create -f</code>	Creates resource once (fails if exists)
<code>kubectl apply -f</code>	Creates OR updates resource
Best practice	Always use <code>apply</code>

You can literally say:

create is imperative management, apply is declarative management.

TASK-8 — Edit Running Pod (`kubectl edit`)

8.Edit an Existing Pod Configuration Task: Use `kubectl edit pod <pod_name>` to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using `kubectl exec`.

Now we will directly modify a live pod without touching YAML file.

Step 1 — Open live configuration

kubectl edit pod testpod
It opens editor.

```
    { apiVersion: v1, kind: Pod, metadata: { annotations: {}  
creationTimestamp: "2026-02-17T10:32:21Z"  
name: testpod  
namespace: default  
resourceVersion: "14586"  
uid: 7d573bf0-ccc0-4a26-8e6f-34f08acdea72  
spec:  
  containers:  
  - image: nginx:alpine  
    imagePullPolicy: IfNotPresent  
    name: nginx  
    resources: {}  
    terminationMessagePath: /dev/termination-log  
    terminationMessagePolicy: File  
    volumeMounts:  
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount  
      name: kube-api-access-p97br  
      readOnly: true  
  dnsPolicy: ClusterFirst
```

```
    cni.projectcalico.org/podIP: 192.168.235.134/32  
    cni.projectcalico.org/podIPs: 192.168.235.134/32  
    kubectl.kubernetes.io/last-applied-configuration: |  
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"testpod"  
creationTimestamp: "2026-02-17T10:32:21Z"  
name: testpod  
namespace: default  
resourceVersion: "14586"  
uid: 7d573bf0-ccc0-4a26-8e6f-34f08acdea72  
spec:  
  containers:  
  - image: nginx:1.27  
    imagePullPolicy: IfNotPresent  
    name: nginx  
    resources: {}  
    terminationMessagePath: /dev/termination-log  
    terminationMessagePolicy: File  
    volumeMounts:  
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount  
      name: kube-api-access-p97br  
      readOnly: true  
  dnsPolicy: ClusterFirst  
  enableServiceLinks: true  
  nodeName: worker1  
  preemptionPolicy: PreemptLowerPriority  
  priority: 0  
  restartPolicy: Always  
  schedulerName: default-scheduler  
  securityContext: {}  
  serviceAccount: default  
  serviceAccountImagePullSecrets: default
```

Step 2 — Change container image

Find this line:

```
image: nginx:alpine
```

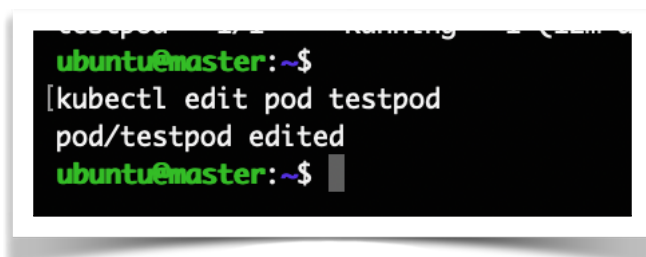
Change to:

```
image: nginx:1.27
```

Save & exit:

ESC → :wq → Enter

Kubernetes will automatically restart the container.



Step 3 — Validate change

Run:

```
kubectl describe pod testpod | grep Image
```

Expected:

```
Image: nginx:1.27
```

```

ubuntu@master:~$ kubectl describe pod testpod | grep Image
Image:          nginx:1.27
Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
Normal Pulled   23m          kubelet        Successfully pulled image "nginx:1.25" in 2.483s (2.484s including waiting). Image size: 71005258 bytes.
Normal Pulled   20m          kubelet        Successfully pulled image "nginx:alpine" in 1.39s (1.39s including waiting). Image size: 25987617 bytes.
Normal Pulled   4m28s        kubelet        Successfully pulled image "nginx:1.27" in 2.291s (2.291s including waiting). Image size: 72406859 bytes.
ubuntu@master:~$

```

Step 4 — Extra proof

`kubectl logs testpod`

```

ubuntu@master:~$ kubectl logs testpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/17 10:51:30 [notice] 1#1: using the "epoll" event method
2026/02/17 10:51:30 [notice] 1#1: nginx/1.27.5
2026/02/17 10:51:30 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
2026/02/17 10:51:30 [notice] 1#1: OS: Linux 6.14.0-1018-aws
2026/02/17 10:51:30 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/17 10:51:30 [notice] 1#1: start worker processes
2026/02/17 10:51:30 [notice] 1#1: start worker process 30
2026/02/17 10:51:30 [notice] 1#1: start worker process 31
ubuntu@master:~$

```

New nginx startup logs appear → means pod recreated.

TASK-9 — Expose Pod using ClusterIP Service

9. Expose a Pod Using a Service Task: Create a YAML file to expose your firstpod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using `kubectl get svc`.

Step 1 — Create firstpod (if not exists)

```
kubectl run firstpod --image=nginx:latest --restart=Never
```

Check:

```
kubectl get pods
```

```
2026/02/17 10:51:30 [notice] 1#1: start worker process 31
ubuntu@master:~$ kubectl run firstpod --image=nginx:latest --restart=Never
pod/firstpod created
ubuntu@master:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
firstpod  1/1     Running   0           7s
testpod   1/1     Running   2 (14m ago) 33m
ubuntu@master:~$ kubectl get pod firstpod --show-labels
NAME      READY   STATUS    RESTARTS   AGE   LABELS
firstpod  1/1     Running   0           19s   run=firstpod
ubuntu@master:~$
```

Step 2 — Check label of pod

```
kubectl get pod firstpod --show-labels
```

You will see something like:

```
run=firstpod
```



```
firstpod 1/1 Running 2 (21m ago) 50m
ubuntu@master:~$ kubectl get pod firstpod --show-labels
NAME      READY   STATUS    RESTARTS   AGE   LABELS
firstpod  1/1     Running   0          19s   run=firstpod
ubuntu@master:~$
```



Important:

Service selector **MUST** match this label.

Step 3 — Create Service YAML

Vi firstpod-service.yaml

```
[ubuntu@master:~$ vi firstpod-service.yaml
[ubuntu@master:~$ cat firstpod-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: firstpod-service
spec:
  selector:
    run: firstpod
  type: ClusterIP
  ports:
  - port: 80
    targetPort: 80

ubuntu@master:~$
```

Step 4 — Apply Service

kubectl apply -f firstpod-service.yaml

```
target 0.0.0.0:80

ubuntu@master:~$ kubectl apply -f firstpod-service.yaml
service/firstpod-service created
ubuntu@master:~$
```

Step 5 — Verify (MAIN ANSWER)

```
kubectl get svc
```

```
service/firstpod-service created
[ubuntu@master:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
firstpod-service    ClusterIP   10.105.123.82 <none>       80/TCP           31s
kubernetes          ClusterIP   10.96.0.1     <none>       443/TCP          23h
nginx-service       NodePort    10.107.18.212 <none>       80:32571/TCP     22h
ubuntu@master:~$
```

```
[ubuntu@master:~$ curl 192.168.189.70
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

TASK-10: Add CPU & Memory limits

10. Pod with Resource Limits and Requests Task: Add resource requests and limits to the containers in your YAML file. Specify CPU and memory requests/limits for both containers and deploy the Pod. Use `kubectl describe pod` to verify if the resource configurations are correctly applied.

Step 1 — Create YAML

Vi resourcepod.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: resourcepod
spec:
  containers:
    - name: nginx1
      image: nginx
      ports:
        - containerPort: 80
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"
    - name: nginx2
      image: nginx
      command: ["/bin/sh", "-c"]
      args:
        - sed -i 's/80/81/g' /etc/nginx/conf.d/default.conf && nginx -g 'daemon off;'
      ports:
        - containerPort: 81
      resources:
        requests:
          memory: "64Mi"
          cpu: "200m"
        limits:
          memory: "128Mi"
          cpu: "400m"
```

Step 2— Apply

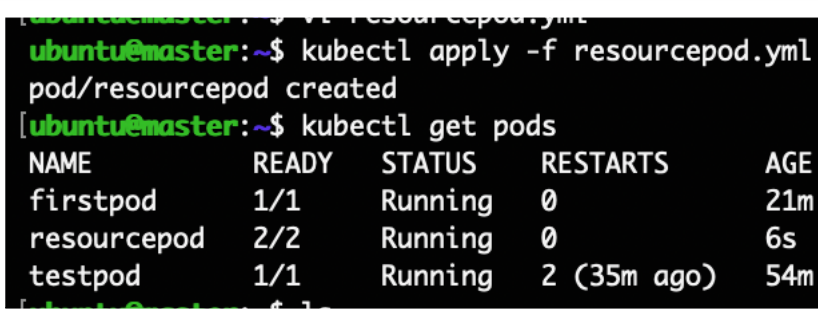
```
kubectl apply -f resourcepod.yml
```

—> Check:

```
kubectl get pods
```

—> Now should be:

```
resourcepod    2/2    Running
```

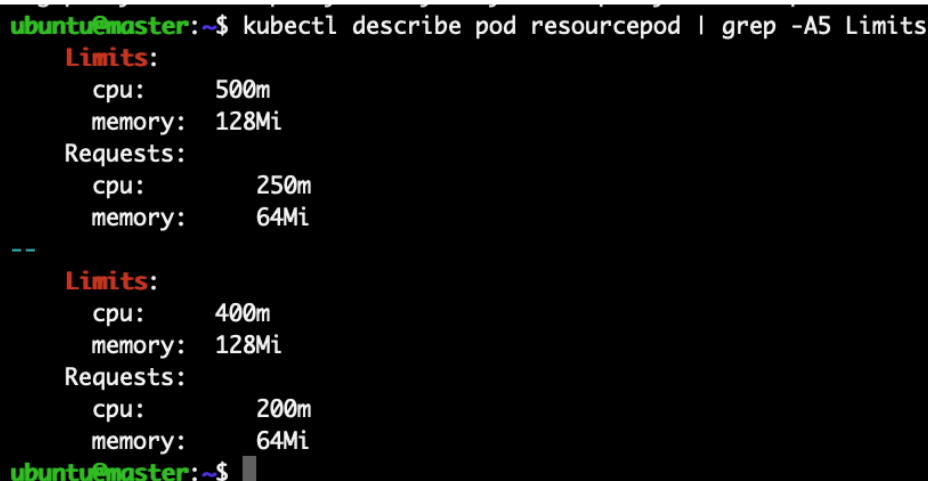


```
ubuntu@master:~$ kubectl apply -f resourcepod.yml
pod/resourcepod created
ubuntu@master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
firstpod	1/1	Running	0	21m
resourcepod	2/2	Running	0	6s
testpod	1/1	Running	2 (35m ago)	54m

Step 3 — Final verification

```
kubectl describe pod resourcepod | grep -A5 Limits
```



```
ubuntu@master:~$ kubectl describe pod resourcepod | grep -A5 Limits
```

Limits:

cpu:	500m
memory:	128Mi

Requests:

cpu:	250m
memory:	64Mi

--

Limits:

cpu:	400m
memory:	128Mi

Requests:

cpu:	200m
memory:	64Mi

```
ubuntu@master:~$
```

